Mesurer son application

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Tests et qualité **3**

A. Introduction 3

B. Qu'est-ce que la qualité logicielle ? 3

C. Indicateurs de qualité logicielle 4

D. Sous-caractéristiques qualité 5

II. Exercice : Quiz **6**

III. Travailler la qualité en amont **6**

A. Préparer la qualité 6

B. Objectifs SMART 7

IV. Exercice : Quiz **8**

V. Tester, toujours tester **8**

A. Différents niveaux de tests 8

B. Types de tests 9

C. Favoriser la détection des anomalies au plus tôt 11

VI. Exercice : Quiz **11**

VII. Améliorer la qualité d'un logiciel **12**

A. Origines de la non-qualité 12

B. Agir sur la qualité 13

C. Versionning 13

VIII. Exercice : Quiz **14**

IX. Essentiel **15**

X. Auto-évaluation **16**

A. Exercice : 16

B. Test 16

Solutions des exercices **17**

Tous droits réservés à STUDI - Reproduction interdite

I. Tests et qualité

Contexte

Vous développez votre application et vous souhaitez connaître son état de santé, son niveau de qualité. Vous décidez alors de mesurer un certain nombre d'indicateurs qui vont vous donner de précieux renseignements.

Nous allons voir ensemble lors de ce cours les indicateurs qu'il est possible de mesurer, et ce que signifie la qualité dans le domaine des logiciels.

A. Introduction

La qualité est une notion en grande partie subjective. Elle peut signifier différentes choses pour différentes personnes. Mesurer la qualité se fait nécessairement sur la base d'indicateurs, que l'on appelle aussi indicateurs de performance. Savez-vous quels indicateurs clés suivre lorsque vous souhaitez mesurer la qualité d'un logiciel ?

Le nombre d'anomalies est bien entendu un indicateur incontournable. Celui-ci peut être relevé à différents moments et à différents niveaux. On pourrait par exemple s'intéresser de près aux nombres d'anomalies relevées suite à un nouveau déploiement en environnement de production. La qualité ne se mesure pas cependant par rapport à ce seul indicateur, il en existe une grande quantité. Et d'abord, qu'est-ce que la qualité logicielle ?

B. Qu'est-ce que la qualité logicielle ?

La qualité logicielle est une appréciation, qui est portée sur une application de manière globale. Elle se base sur différents indicateurs. Elle vise à mesurer le degré avec lequel cette application répond à des exigences.

Selon votre propre expérience, vous êtes probablement déjà familier avec certains :

- **La complétude des fonctionnalités :** est-ce que le produit répond bien à la demande ? A-t-on adressé l'ensemble du besoin ou avons-nous oublié des éléments ?
- **La capacité à corriger les défauts et apporter des évolutions :** peut-on adapter rapidement le logiciel en cas de nécessité ?
- **La fiabilité :** peut-on avoir confiance dans l'utilisation de l'outil et les données qu'il nous retourne ?
- **La tolérance aux pannes :** est-ce que le logiciel sera toujours opérationnel après une défaillance technique comme le crash d'un disque dur, d'un serveur ou d'une connexion réseau ? Après une coupure d'électricité ?
- **La facilité d'utilisation :** les utilisateurs comprennent-ils facilement comment utiliser le logiciel ?
- **La compatibilité :** le logiciel est-il bien compatible avec les outils et les équipements déjà en place dans l'organisation ?
- **La performance :** les temps de réponse et les délais de traitement sont-ils satisfaisants ?
- Etc.

Un logiciel est un produit dont la fiabilité n'est ni prédictible ni observable, comme cela serait le cas sur un produit matériel qui sort d'usine. Les anomalies dans un logiciel sont généralement présentes de manière **latente** : certaines peuvent survenir rapidement et d'autres jamais, bien qu'un réel défaut de conception existe dans l'application.

La qualité d'un logiciel dépend très fortement de la manière dont il est construit, et des processus employés pour son développement. Pour ces raisons, il est essentiel d'être rigoureux et d'adopter de bonnes pratiques le plus tôt possible.

Tous droits réservés à STUDI - Reproduction interdite

C. Indicateurs de qualité logicielle

La norme ISO 25010 définit **six catégories** d'indicateurs :

La capacité fonctionnelle

C'est la capacité du logiciel à répondre, à travers ses différentes fonctionnalités, aux besoins et aux exigences explicites **ou implicites** des utilisateurs. Les besoins implicites peuvent concerner la conformité vis-à-vis de certaines normes (sécurité, RGPD, etc.), mais aussi des besoins de précision, d'interopérabilité, de disponibilité. Faites attention à ces besoins : ils ne sont jamais clairement exprimés par le client mais ils existent bel et bien. Faites-le parler pour ne pas passer à côté !

La facilité d'utilisation

Elle porte sur l'effort nécessaire pour apprendre à utiliser le logiciel. Le logiciel doit être facile à comprendre, facile à apprendre, facile à exploiter. Il doit aussi être robuste : une utilisation incorrecte ne devrait pas entraîner de dysfonctionnement.

La fiabilité

Indépendamment des conditions d'exploitation, le logiciel doit rendre des résultats corrects. C'est cette capacité à délivrer le bon résultat même en cas de pannes (logicielles ou matérielles) qui rend le logiciel fiable. Selon le type de projet sur lequel vous travaillez, la tolérance aux pannes peut être un critère très important.

La performance

C'est le rapport entre les ressources utilisées et les résultats délivrés. Les temps de réponse, le débit ou encore l'extensibilité sont des critères de performance. Le logiciel peut-il maintenir ses niveaux de performance même lorsque de nombreux utilisateurs s'y connectent en même temps ? À quel point les ressources sont-elles utilisées ? Est-ce normal ?

La maintenabilité

La maintenabilité, c'est la mesure de l'effort nécessaire pour corriger ou transformer le logiciel. Imaginez ce que serait de naviguer au sein d'un code chaotique et non documenté où chaque changement prendrait énormément de temps pour être appliqué. Cela coûterait très cher à maintenir ! Un logiciel qui n'est pas, ou peu maintenable n'est jamais une bonne affaire pour une entreprise.

Exemple

Une activité critique et bien précise au sein de l'entreprise (par exemple, la gestion des stocks) est assurée via un « *développement maison* », qui a fait ses preuves depuis 20 ans. Le concepteur est toujours dans l'entreprise et c'est lui qui a toujours mis à jour, seul, le logiciel. Un jour, cette personne part à la retraite. L'entreprise souhaite apporter un changement dans le logiciel, mais personne ne sait comment faire. Le logiciel n'est pas maintenable. De plus, si un dysfonctionnement apparaît, toute la gestion des stocks pourrait être perdue par l'entreprise, pour une durée indéterminée. Ce qui n'est évidemment pas acceptable, et même si le logiciel donne entière satisfaction dans le moment présent.

La portabilité

C'est la capacité du logiciel à fonctionner dans un environnement différent de l'environnement initial, aussi bien matériel que logiciel.

Le logiciel doit pouvoir être installé et configuré facilement dans un nouvel environnement.

Attention

Les différents indicateurs peuvent être complémentaires mais aussi **conflictuels.**

Exemple de conflit : une augmentation de la capacité fonctionnelle pourrait avoir un impact **négatif** sur la performance, la maintenabilité et la fiabilité → on ajoute davantage de fonctionnalités mais en contrepartie, le logiciel perd en performance et il est plus lourd à mettre à jour.

Tous droits réservés à STUDI - Reproduction interdite

Exemple de complémentarité :

L'augmentation de la fiabilité, de la disponibilité ou de la maintenabilité peut avoir un impact **positif** sur l'utilisabilité.

D. Sous-caractéristiques qualité

Avec l'objectif d'aller encore plus loin, chaque caractéristique de la norme 25010 vue dans le paragraphe précédent contient aussi des **sous-caractéristiques** que nous allons présenter.

Ces sous-caractéristiques sont au nombre de 27, que vous pouvez librement utiliser pour analyser la qualité globale de votre application.

- **Capacité fonctionnelle :** *le logiciel répond-il aux besoins fonctionnels exprimés ?*
  - Pertinence
  - Exactitude
  - Sécurité
  - Interopérabilité
  - Conformité
- **Fiabilité :** *est-ce que le logiciel maintient sa qualité de service dans la durée ?*
  - Maturité (peu d'incidents déclarés)
  - Tolérance aux pannes
  - Facilité de récupération (pouvoir retourner dans un état opérationnel complet après un incident)
- **Utilisabilité :** *le logiciel demande-t-il peu d'efforts pour pouvoir être utilisé ?*
  - Facilité d'apprentissage
  - Facilité de compréhension
  - Facilité d'exploitation
- **Rendement et efficacité :** *au regard de l'ensemble des exigences, est-ce que le logiciel demande un dimensionnement rentable de sa plate-forme d'hébergement ?*
  - Efficacité dans l'utilisation des ressources : mémoire vive, stockage, processeur, réseau
  - Efficacité des traitements : temps de réponse, taux de transactions
- **Maintenabilité :** *des évolutions et des correctifs peuvent-ils être facilement intégrés au logiciel ?*
  - Facilité pour retrouver l'origine des anomalies
  - Facilité d'analyse du logiciel
  - Facilité pour modifier l'existant
  - Stabilité du logiciel
  - Testabilité du code
- **Portabilité :** *le logiciel peut-il être transféré d'un environnement à un autre ?*
  - Facilité d'adaptation
  - Facilité d'installation
  - Coexistence
  - Indépendance
  - Interchangeabilité

Tous droits réservés à STUDI - Reproduction interdite

Exercice : Quiz

Question 1

Quel indicateur qualité se rattache à la tolérance aux pannes ?

- La maintenabilité
- La portabilité
- La fiabilité
- L'utilisabilité

Question 2

Votre logiciel doit migrer d'une plateforme à une autre afin de gagner en performance. Quel est le critère qualité concerné ?

- La maintenabilité
- La portabilité
- La capacité fonctionnelle

Question 3

Vous mesurez le nombre de crashs par utilisateur et par heure d'utilisation. Quel est le critère qualité concerné ?

- La fiabilité
- La performance
- La facilité d'exploitation

Question 4

Remplir de hautes exigences pour l'ensemble des critères qualité ne peut être que bénéfique.

- Vrai
- Faux

Question 5

L'augmentation de la fiabilité peut-elle avoir un impact positif sur l'utilisabilité ?

- Oui
- Non

II. Travailler la qualité en amont

A. Préparer la qualité

Il est judicieux de mettre en place un **processus qualité** dès le démarrage d'un nouveau projet si l'on veut espérer délivrer un produit de qualité. Si votre organisation n'en possède pas, il serait alors opportun **d'écrire un plan qualité.** Il référencera tous les critères qualité (et les objectifs) que vous souhaitez imposer aux développements. Ce plan qualité sera en quelque sorte un guide et un recueil de bonnes pratiques pour tous les développements de votre organisation.

Tous droits réservés à STUDI - Reproduction interdite

Conseil

Chaque organisation est différente, chaque projet est différent, c'est pourquoi il n'est pas possible de fixer à l'avance des critères et des niveaux d'exigences qui fonctionneront à coup sûr pour tel ou tel travail de développement. Inspirez-vous librement de la liste d'indicateurs fournie au premier paragraphe de ce cours et gardez ceux qui sont les plus pertinents dans votre contexte.

De manière générale :

1. Assurez-vous que le besoin ait été correctement défini par votre client (qui éventuellement, peut être votre patron). Il doit être concis, compréhensible et comporter des critères d'acceptance clairs. Si le besoin est d'avoir un logiciel rapide, cela n'est pas acceptable. Qu'est-ce que veut dire rapide ? Pour qui ? Pour quelles actions ? Dans quel environnement ? Sous quelles conditions ? Quelle est la métrique à utiliser ? Comment pourrait-on vérifier que l'on a bien respecté cette exigence ?
2. Assurez-vous que ce besoin soit partagé par toutes les personnes participant au projet, et que toutes possèdent la même vision. Posez des questions, reformulez, paraphrasez. Il est important que tout le monde partage la même vision avant de commencer le développement. Si vous travaillez dans une petite structure, il y a de fortes chances pour qu'il n'y ait pas de chef de projet et ce travail d'orchestration retombe bien souvent sur les développeurs. Soyez vigilants, faites écrire (et valider !) ce que l'on vous demande de faire. Le tout doit être mesurable, les objectifs doivent être SMART.
3. Testez le code au fur et à mesure de son élaboration. Faites des revues de code, écrivez et lancez des tests unitaires. Travaillez en intégration continue. Ayez un pipeline de livraison logicielle fluide et une communication solide avec les utilisateurs.
4. N'oubliez pas, les tests unitaires doivent être écrits tout au long du développement du logiciel et exécutés à chaque fourniture d'une nouvelle version !
5. Testez, testez, et testez !

B. Objectifs SMART

Afin de vous aider à avoir des objectifs acceptables sur votre projet de développement, vous pouvez vous aider de la méthode SMART.

Un objectif SMART est un objectif qui est **S**pécifique, **M**esurable, **A**ssignable, **R**éaliste et **T**emporellement défini.

- **Spécifique**

  L'objectif doit se rapporter à un élément précis, clair et sans ambiguïté. La précision apportée à la construction de l'objectif vous permettra de facilement mesurer les avancées.

- **Mesurable**

  L'objectif doit être mesurable afin de pouvoir contrôler et valider le résultat.

- **Assignable**

  Il faut impérativement qu'une ou plusieurs personnes soient clairement identifiées comme responsables de la réalisation de l'objectif.

- **Réaliste**

  Il faut faire la différence entre une situation idéale et une situation qui peut être atteinte au sein de votre organisation. L'objectif peut inciter les personnes impliquées à se surpasser, mais il ne doit pas être hors de portée, ou en dessous de leur niveau de performance. Des objectifs trop élevés ou trop quelconques perdent de leurs sens, car ils finissent par être ignorés.

Tous droits réservés à STUDI - Reproduction interdite

- **Temporellement défini**

  Sans repère temporel, l'objectif perdrait toute sa valeur, car il serait vite éclipsé par d'autres demandes plus urgentes. Il faut donc fixer un échéancier clair, en créant éventuellement des jalons intermédiaires qui stimulent les efforts et la motivation.

Exercice : Quiz

Question 1

Un objectif SMART est un objectif :

- ○ Spécifique, Minutieux, Approfondi, Réaliste, Temporellement défini
- ○ Simple, Minutieux, Automatique, Assignable, Temporellement défini
- ○ Spécifique, Mesurable, Assignable, Réaliste, Temporellement défini

Question 2

« *L'équipe doit produire 1 000 lignes de codes et 20 tests unitaires pour la prochaine release du 30 juin* ». Est-ce un objectif SMART ?

- ○ Vrai
- ○ Faux

Question 3

Votre chef vous demande de concevoir un logiciel fiable. Que lui répondez-vous ?

- ○ Oui bien sûr, ce sera fait !
- ○ Fiable n'est pas un objectif SMART

Question 4

« *Le logiciel doit être utilisable par Jean-Michel d'ici vendredi prochain* ». Est-ce un objectif SMART ?

- ○ Vrai
- ○ Faux

IV. Tester, toujours tester

A. Différents niveaux de tests

Les tests logiciels se font à travers plusieurs niveaux. On en distingue communément quatre.

Les tests unitaires

Ils sont menés par le développeur lui-même. Ces tests consistent à vérifier la bonne exécution des fonctions dont le développeur a la charge. Elles sont testées à ce stade de manière indépendante, la plupart du temps avec des données réduites et avec des bouchons (pour pallier des interfaces qui n'existent pas sur le poste du développeur). Il est très utile de définir au plus tôt des jeux de tests représentatifs, et de faire en sorte que le développeur les entretienne. Les tests unitaires doivent être construits et joués tout au long du développement du logiciel.

Les tests d'intégration

L'objectif des tests d'intégration est de tester la **mise en commun** de plusieurs composants. Ils visent à tester aussi l'enchaînement de processus complets. Ces tests ne se limitent donc pas aux simples fonctions unitaires. On parle également de tests « *en boîte blanche* », car les testeurs regardent (et peuvent regarder) les relations entre les

Tous droits réservés à STUDI - Reproduction interdite

composants directement au niveau du code. Cela leur permet de vérifier la bonne coopération des composants entre eux. Pour être efficaces, les tests d'intégration doivent être pensés et mis à disposition (via des jeux de test, un cahier des recettes), le plus tôt possible dans le cycle de développement.

Ils sont généralement confiés à un testeur dédié au sein de l'équipe de développement, qui travaille en étroite collaboration avec d'autres experts (urbaniste, chef de projet, autres développeurs, etc.), étant donné la complexité qui peut ressortir de ce type de tests.

Les tests de qualification

Les tests de qualification ne se préoccupent pas de l'implémentation des composants au sein du logiciel. Ils visent à tester les différents cas d'**utilisation** (via l'ensemble des fonctionnalités du logiciel) en utilisant des scénarios complets et déroulés méthodiquement. On parle de tests « *en boîte noire* », car les testeurs ne peuvent pas regarder le code de l'application ou les relations entre les différents composants, ils ne peuvent que constater un résultat.

Il est recommandé que les testeurs impliqués dans des tests de qualification ne soient pas les mêmes personnes qui développent le logiciel. Cela permet de ne pas biaiser les tests et les interprétations qui pourraient en être faites. Les tests de qualification permettent de valider la non-régression du logiciel (pas d'introduction d'anomalies sur des aspects qui étaient opérationnels dans la version précédente), en passant à une nouvelle version.

Les tests d'acceptation (recette utilisateur)

Les tests d'acceptation sont menés avec des utilisateurs pilotes, qui sont généralement porteurs du besoin initial et qui sont hautement motivés par les résultats que délivrera le logiciel. Ces utilisateurs valident donc l'adéquation du logiciel, avec leur métier et leurs besoins. Il est important d'associer très tôt vos futurs utilisateurs au projet (et d'identifier des utilisateurs pilotes), car cela vous permet de mieux cerner leurs attentes et leurs critères d'acceptation. Ces derniers ne sont pas toujours exprimés spontanément au début du projet et il est possible que vous en découvriez de nouveaux, lors de la recette utilisateur. Choisissez des utilisateurs motivés afin d'en faire de précieux alliés le plus tôt possible.

Conseil

Un test que vous déroulez manuellement représente pour vous, une charge de travail à chaque fois que vous l'exécutez. Par contre, si votre test a été consolidé (scripté) alors le temps passé à le préparer devient un investissement, car vous pouvez le rejouer à l'infini sans que cela n'impacte votre charge de travail. Automatiser les tests unitaires, et certains tests d'intégration, font partie de l'intégration continue.

Conseil

La documentation est essentielle pour un logiciel, et elle ne doit pas se limiter à des guides utilisateurs ou à des guides de dépannage. Une bonne maintenance dépend directement de la qualité des documents et de leur couverture. Par exemple, le code des tests unitaires, les fichiers de configuration associés, les scénarios de tests et les jeux de données y participent grandement. La documentation est souvent, à tort, reléguée en arrière-plan. Cela porte atteinte à la qualité du logiciel (ex : sur la maintenabilité).

B. Types de tests

Plusieurs grandes catégories de tests peuvent être identifiés :

Les **tests fonctionnels**

Ils valident (ou invalident) le **service rendu** par le logiciel à travers ses différentes fonctionnalités, et son comportement. Ce type de test est généralement celui pour lequel le plus grand investissement est apporté. Il est étroitement lié aux spécifications, qui détaillent la manière dont le logiciel doit fonctionner. Pour des tests fonctionnels efficaces, il est utile de travailler avec vos futurs clients à la rédaction des scénarios de tests En effet, même si vous (ou l'équipe de développement) possédez une bonne connaissance des besoins de vos clients, ce sont eux qui restent experts dans leur métier. Ils vous seront d'une aide précieuse afin d'éprouver les fonctionnalités du logiciel. Ces tests peuvent être écrits dès que les spécifications sont arrêtées, c'est-à-dire relativement tôt dans la vie du projet.

Tous droits réservés à STUDI - Reproduction interdite

Les **tests de compatibilité**

Il faut à minima vérifier que le logiciel s'installe bien sur l'environnement cible (système d'exploitation, plugins, paramétrages, bases de données etc.) et qu'il se lance normalement, c'est-à-dire qu'il n'y a aucune erreur au démarrage. Les tests de compatibilité mettent en lumière le respect (ou le non-respect) des bonnes pratiques afin de limiter les adhérences du logiciel avec son environnement direct. Cela peut inclure des machines virtuelles, l'allocation de ressources, le fait de pouvoir lancer le logiciel sans droits d'administration etc. Attention, il ne faut pas confondre l'environnement cible du logiciel avec le poste informatique des développeurs, sinon voici la phrase que vous entendrez à coup sûr : « *Je ne comprends pas, ça fonctionne chez moi !* ». Il est aussi judicieux de déployer des mises en production à blanc, afin de valider le fait que le logiciel s'installera bien le jour J, et que les personnes en charge des opérations maîtrisent la documentation et les processus.

Les **tests de robustesse**

Il est fort probable qu'un jour, le logiciel rencontre une situation limitée ou dégradée (ex : perte d'une connexion réseau, forte latence, etc.). Les tests de robustesse ne sont pas toujours déroulés, ce qui est dommage, car ils ne sont pas compliqués à mettre en œuvre et vous rassurent sur le comportement de l'application. Vous pouvez par exemple débrancher un câble, arrêter brutalement une machine, couper l'électricité, observer le résultat et ce qui se passe quand la situation revient à la normale.

Il est toujours utile de savoir comment le logiciel réagit dans de tels cas. Cela vous aidera aussi à définir des scénarios de reprise. Une fois l'électricité revenue ou le câble rebranché, est-ce que le logiciel retourne automatiquement en situation nominale ou une action humaine est-elle requise ? Faire passer des tests de robustesse à un logiciel ne représente pas une grande charge, alors que les conséquences d'une situation dégradée sont généralement impossibles à évaluer tant qu'elles ne se sont pas produites. Prévoyez comment le logiciel doit réagir dans le cas d'une situation dégradée, et vous améliorerez sa qualité !

Les **tests de performance**

Ils sont souvent menés vers la fin du projet, car il n'y a pas d'intérêt particulier à tester la performance d'un logiciel qui n'est ni abouti ni optimisé. Les facteurs qui agissent sur la performance sont nombreux, et il est préférable d'utiliser l'environnement cible pour les dérouler. En prévision de tests de performance complets à la fin du projet, les développeurs peuvent toutefois très bien tester le comportement de certains services spécifiques. Cela permet de limiter les risques, quant à de mauvais choix techniques, qui dégraderaient les performances au-delà du seuil acceptable. Les résultats de ces tests (comme pour tous les tests) doivent être bien suivis et documentés. En formalisant les conditions dans lesquelles le logiciel a été testé et les résultats que vous obtenez, vous enrichissez une base de connaissances très utile, qui servira de base aux optimisations.

Les **tests de montée en charge**

Ce ne sont pas des tests de performance à proprement parler. Un logiciel peut en effet être performant, mais incapable de soutenir une augmentation du nombre de ses utilisateurs. Un logiciel, et plus particulièrement un site internet ou une application web, doit être conçu pour pouvoir monter en charge très facilement via l'ajout de ressources matérielles et de manière quasiment illimitée. Imaginez qu'un site d'e-commerce soit hors-service à chaque pic de connexions. Cela serait désastreux pour les ventes et pour l'image de marque ! Les tests de montée en charge sont réalisés avec des outils permettant de simuler un parcours utilisateur, multiplié par le nombre d'instances souhaitées. Vous pourrez lier tests de montée en charge et tests de performance afin de savoir comment le logiciel répond en cas de fortes sollicitations, et prendre d'éventuelles actions préventives.

Les **tests d'ergonomie**

L'ergonomie est un sujet très souvent mis de côté dans les développements, car elle nécessite des profils particuliers (UI / UX designers) que l'on ne trouve pas dans toutes les entreprises. Par conséquent, les tests d'ergonomie sont aussi souvent mis de côté alors que l'ergonomie est un des facteurs qualité les plus importants chez les utilisateurs de logiciels. C'est la qualité qui est ressentie immédiatement lors de la prise en main. Les tests d'ergonomie visent le « *Look & Feel* », c'est-à-dire aussi bien l'apparence visuelle que l'expérience de l'utilisateur (la facilité de compréhension et d'utilisation).

Tous droits réservés à STUDI - Reproduction interdite

C. Favoriser la détection des anomalies au plus tôt

Le coût d'une anomalie n'est pas le même selon le moment où elle est introduite et détectée dans le logiciel. Il est évident que si le développeur détecte une anomalie juste après avoir écrit son code, l'impact est très faible et son coût est nul ou presque. Cela demandera juste un peu de temps supplémentaire dédié au debug.

Plus l'anomalie est détectée tard et plus elle coûtera chère à réparer. Il faut donc avoir le moins d'anomalies possible après une mise en production, d'où l'intérêt de mettre au point toute une batterie de tests en amont. Pensez aux constructeurs automobiles qui parfois rappellent des milliers de véhicules après commercialisation, à cause d'un défaut non détecté en usine ! Le coût est ici maximal.

Adoptez donc cette bonne pratique, et faites en sorte que le logiciel soit de qualité le plus tôt possible, et surtout avant sa mise en production.

Exercice : Quiz

Question 1

Lequel de ces tests est réalisé « *en boîte noire* » ?

- O Le test unitaire
- O Le test de qualification
- O Le test d'intégration
- O Le test d'acceptation

Question 2

Un test consolidé dans un script est :

- O Une charge
- O Un investissement

Question 3

La service rendu par le logiciel est vérifié par :

- O Des tests de robustesse
- O Des tests de compatibilité
- O Des tests fonctionnels

Question 4

Les tests de performance sont réalisés à la fin du projet.

- O Vrai
- O Faux

Question 5

Une mise en production à blanc permet de s'assurer de :

- O La robustesse du logiciel
- O La compatibilité du logiciel
- O La bonne adéquation du logiciel avec les besoins fonctionnels

Tous droits réservés à STUDI - Reproduction interdite

V. Améliorer la qualité d'un logiciel

A. Origines de la non-qualité

On trouve très souvent deux origines à un manque de qualité dans un logiciel :

- Le **manque de formation** ou de **sensibilisation** aux activités de tests et de qualité dans l'ensemble. Cela peut se traduire par trop peu ou pas d'écriture et d'exécution de tests unitaires pendant le développement. Les tests n'ont pas bonne réputation, ils passent parfois pour inutiles, superflus, ou étant à dérouler par quelqu'un d'autre (« *moi je fais ce qu'on me demande, je développe. Je n'ai pas de temps pour tester et quelqu'un d'autre le fera bien. De toute façon, on me le dira forcément si quelque chose ne va pas. En attendant, je continue de développer.* ».
- Une période de temps beaucoup trop courte dédiée aux tests manuels et à la recette avec les utilisateurs. En cas de retard, c'est souvent cette tâche qui est sacrifiée. Les tests sont donc bâclés, lorsqu'ils sont faits, afin de mettre en production aussitôt que possible.

Fondamental

Pour savoir comment améliorer la qualité de votre logiciel, il faut tout d'abord vous **fixer des critères** de qualité que vous allez **suivre** (*voir : paragraphe 1*). Ensuite, il faut savoir comment **mesurer** les indicateurs que vous avez choisis, et leur évolution.

Selon la taille de votre organisation et la nature de votre projet, vous n'allez à coup sûr pas utiliser les 27 indicateurs de la norme ISO 25010. Le mieux est l'ennemi du bien, et voici une suggestion d'indicateurs que vous pourriez plus facilement adopter :

Qualité fonctionnelle : le logiciel doit respecter les spécifications sur lesquelles il est basé, et répondre aux besoins des utilisateurs. Combien d'exigences ont été formulées par votre client ? Combien en avez-vous pris en compte et comment pouvez-vous montrer qu'elles ont été respectées ? Votre calcul vous amène au taux de couverture des exigences fonctionnelles.

Fiabilité : le logiciel doit restituer des données cohérentes et résister aux erreurs de l'utilisateur, aux enchaînements non prévus. Ajoutez des contrôles sur les différents champs de formulaire, afin de limiter les erreurs de saisie. Enregistrez et suivez le nombre de tickets d'anomalies. Enregistrez les plantages dans des logs. Vérifiez souvent les logs et dressez l'évolution de ces plantages en fonction du temps. Déroulez régulièrement des campagnes de recette utilisateur et observez l'évolution du nombre d'incidents déclarés. La fiabilité de votre logiciel est-elle en hausse depuis la dernière version ?

Facilité d'utilisation (user-friendly) : les éléments d'interfaces mis en place doivent permettre à l'utilisateur de maîtriser facilement le logiciel, et de savoir trouver l'information qu'il recherche. Vous aviez fixé l'objectif suivant : « *Un nouvel utilisateur, qui n'est pas formé à mon logiciel, doit pouvoir trouver l'information qu'il recherche en moins de 2 minutes et noter son expérience à au moins 7/10* ». Faites appel à des utilisateurs qui n'ont encore jamais vu votre logiciel, et vérifiez si votre objectif est rempli. Mettez en place des actions de feedback (enquêtes par e-mail, téléphone, sondage en ligne, forum, formulaire de contact, etc.).

La performance : vous aviez spécifié que le logiciel doit s'ouvrir en moins de 12 secondes, et chaque changement d'écran ne doit pas prendre plus de 2 secondes. Que se passe-t-il en réalité ? Votre logiciel remplit-il votre critère qualité de performance ?

La maintenabilité : le code du logiciel doit respecter les standards en vigueur, des normes, des bonnes pratiques. Une documentation complète et testée doit aussi permettre de maintenir le logiciel à moindre coût. Quel est le résultat d'une revue de code ? En combien de temps un nouvel administrateur est-il capable d'effectuer une opération de maintenance standard ? Combien de thèmes doivent être abordés dans la documentation, et combien sont couverts ? Une personne qui ne connaît pas le projet, mais qui obtient toute la documentation, sait-elle prendre le logiciel en main ?

Tous droits réservés à STUDI - Reproduction interdite

Tous ces indicateurs doivent être suivis de manière régulière pour être utiles. Il pourrait suffire de placer un rendez-vous périodique dans votre agenda ou d'instaurer un rituel avec votre équipe. Le tableau Excel est un très bon moyen pour centraliser les indicateurs.

B. Agir sur la qualité

Il existe plusieurs actions que vous pourriez mettre en place pour améliorer la qualité au niveau de chaque indicateur.

Voici quelques exemples :

Complexité du code

Analyser votre code (ou mieux : faites analyser votre code) pour trouver les méthodes complexes à refactoriser.

Taux de couverture des tests unitaires

Augmentez votre couverture de tests unitaires en en écrivant de nouveaux.

Nombre de tests validés

Faites-en sorte que tous les tests réussissent. Si ce n'est pas le cas, agissez en conséquence pour corriger la situation.

Temps d'exécution des tests

Utilisez un serveur d'intégration continue afin de pouvoir lancer des tests de manière automatique. Ceci fera gagner énormément de temps aux équipes de développement.

Délai d'ouverture du logiciel

Mettez en place un tracking afin de savoir en combien de temps votre logiciel se lance. Prenez en compte les différents environnements de vos utilisateurs afin d'identifier d'éventuelles optimisations à apporter.

Couverture des fonctionnalités

Créez une matrice de respect des exigences : lister les fonctionnalités demandées par votre client, puis vérifiez si elles sont implémentées et respectées.

C. Versionning

Le versionning est le fait d'attribuer un code unique à chaque version du logiciel. Cette gestion des versions est cruciale au sein d'un projet, car elle permet d'identifier et de suivre les modifications effectuées au fur et à mesure sur le logiciel.

Chaque fait technique (anomalie ou évolution) est identifié par rapport à un numéro de version.

- Telle anomalie a été détectée dans la version 1.1,
- Tel correctif a été déployé dans la version 1.2,
- Telle évolution a été implémentée dans la version 1.4.

On observe généralement 3 types de versionning :

1. **Le versionning simple (X.Y)**

   « *X* » représente la **version** au sens « *découpage fonctionnel* » du produit et « *Y* » représente sa **révision**. La toute première version d'un produit porte logiquement le numéro 1.0. Si les fonctionnalités ne changent pas, mais que des correctifs sont apportés, la bonne pratique consiste à incrémenter le numéro de révision, tout en conservant le numéro de version. Dans certains cas, on peut utiliser l'année en cours pour le numéro de version et le mois pour le numéro de révision. Par exemple, 2020.09 (ou 20.09).

Tous droits réservés à STUDI - Reproduction interdite

2. **Le versionning étendu (X.Y.Z)**

   « *X* » représente une version majeure et « *Y* » représente une version mineure. « *Z* » représente le numéro de révision de la version mineure. Le versionning étendu permet donc de différencier les lots d'évolution de manière plus fine. Si votre nouvelle version majeure concerne une évolution intitulée « *import de dossiers* », vous pourriez alors avoir par exemple la version « *2.0.0* » qui embarque les fonctionnalités de base, puis une version « *2.1.0* » qui ajoute l'import de dossiers PDF, la version « *2.2.0* » qui ajoute l'import de dossiers Word, la version « *2.2.1* » qui contient des correctifs.

3. **Mention de l'état de la version (X.Y-état ou X.Y.Z-état)**

   Il est assez courant d'ajouter un « état » à un numéro de version quand elle n'est pas aboutie ou comporte certaines restrictions. L'état peut être :

   - « ***-dev*** » : version en cours de développement, non utilisable.
   - « ***-alpha*** » : version non aboutie (quelques fonctionnalités sont présentes), contient très probablement des anomalies.
   - « ***-beta*** » : version presque aboutie (la plupart des fonctionnalités sont présentes), contient très probablement des anomalies.
   - « ***-rc1*** » : version candidate à une mise en production (*release candidate*) les fonctionnalités sont toutes développées, mais pourraient encore contenir quelques anomalies à corriger.
   - « ***-stable*** » : version terminée et validée, réputée utilisable en production.

La gestion des versions est effectuée par le chef de projet (qui peut être le développeur) et suit une **logique** qu'il détermine en fonction de plusieurs critères.

Par exemple :

- Le type de développement,
- La fréquence des itérations (évolutions / correctifs),
- Le mode de commercialisation,
- La communication.

De nombreux outils existent concernant la gestion de versions comme Git, Mantis, SVN, RedMine.

Le principe de base consiste à **générer** des fiches de faits techniques (via un **ticket**) et à positionner en face, un numéro de version dans laquelle la tâche est identifiée puis réalisée.

Exercice : Quiz

Question 1

Vous souhaitez améliorer la qualité de votre logiciel. Quel est le principal pré-requis ?

- ○ Avoir défini des indicateurs qualité au début du projet
- ○ Disposer d'utilisateurs motivés
- ○ Acheter du matériel performant
- ○ Avoir un bon chef

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Le versionning, c'est :

- Le fait d'archiver en temps et en heure son code source
- Afficher le numéro de version de son logiciel de manière apparente
- Adopter une convention de numérotation pour les versions de son logiciel

Question 3

Laquelle de ces versions est susceptible de comporter le moins d'anomalies ?

- La version alpha
- La version beta
- La version rc1

Question 4

Vous venez de déployer la version 14.1 de votre logiciel en production. On vous demande de travailler sur des évolutions importantes en vue d'une prochaine version, mais aussi sur des correctifs, car plusieurs anomalies sont identifiées. Vous livrez les correctifs. Quel numéro donnez-vous à la version ?

- 15.0
- 15.1
- 14.2

Question 5

Vous désirez faire tester votre logiciel aussi tôt que possible. Vous réunissez pour cela une équipe d'utilisateurs motivés. Quelle est la version qui servira aux tests ?

- Dev
- Alpha
- Beta

VI. Essentiel

Mesurer son application est une activité essentielle lorsque l'on se soucie de sa qualité et de son optimisation. Il est toutefois utopique de vouloir suivre un nombre d'indicateurs tels que vous (ou votre équipe) n'auriez pas le temps de les collecter, les traiter et les adresser convenablement. Posez-vous des questions. Quels sont les besoins (exprimés et sous-entendus) de votre client ? Triez-les par ordre d'importance et définissez des indicateurs qualité pertinents qui vous aideront dans la réalisation de votre mission.

Votre client attache une grande importance à l'ergonomie ? Mettez en place des enquêtes et relevez la satisfaction de vos utilisateurs, notée par exemple sur 5 points.

Vous l'avez compris, on ne mesure pas toutes les applications de la même manière. Il existe cependant des guides et des bonnes pratiques sur lesquels vous pouvez largement vous reposer.

Tous droits réservés à STUDI - Reproduction interdite

VII. Auto-évaluation

A. Exercice :

Vous arrivez sur un projet en cours de spécification. Vous vous rendez compte qu'il n'y a pas de chef de projet clairement désigné et que tout le monde dans l'équipe de développement fonce tête baissée dans les aspects techniques. Les facteurs qualité ne sont pas abordés autrement que par ses termes « *On va faire un logiciel pour permettre à l'usine de suivre son stock. Ils veulent juste pouvoir inventorier leurs produits, ce sera facile* ».

Question 1

Que proposez-vous à l'équipe de développement ?

Question 2

Comment allez-vous mesurer le travail accompli ?

B. Test

Vous arrivez sur un projet en cours de spécification. Vous vous rendez compte qu'il n'y a pas de chef de projet clairement désigné et que tout le monde dans l'équipe de développement fonce tête baissée dans les aspects techniques. Les facteurs qualité ne sont pas abordés autrement que par ses termes « *On va faire un logiciel pour permettre à l'usine de suivre son stock. Ils veulent juste pouvoir inventorier leurs produits, ce sera facile* ».

Exercice : Quiz

Question 1

Des process qualité bien documentés permettent de prédire la fiabilité d'un logiciel une fois développé.

- ○ Vrai
- ○ Faux

Question 2

Parmi les phrases suivantes, laquelle est relative à la fiabilité ?

- ○ Le produit répond parfaitement à la demande du client.
- ○ Le logiciel peut être maintenu facilement par les équipes techniques.
- ○ Les utilisateurs ont confiance dans les traitements effectués par le logiciel.

Question 3

Une utilisation incorrecte de votre logiciel ne doit pas entraîner de dysfonctionnement. On parle ici de :

- ○ Performance
- ○ Robustesse
- ○ Maintenabilité
- ○ Portabilité

Question 4

La maintenabilité, c'est :

- ○ L'aptitude du logiciel à fonctionner dans un nouvel environnement
- ○ La mesure de l'effort nécessaire pour modifier le logiciel
- ○ La capacité du logiciel à délivrer des résultats conformes

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Une augmentation de la capacité fonctionnelle peut-elle avoir un impact négatif sur la performance de votre application ?

- Oui
- Non

Question 6

Il vaut mieux se concentrer sur les tests unitaires au début du projet afin d'en produire le plus possible et être tranquille pour le reste du développement.

- Oui, ce qui est fait n'est plus à faire !
- Non, la rédaction des tests unitaires est une tâche continue

Question 7

Le plan qualité est un document indispensable que possèdent forcément toutes les entreprises informatiques.

- Vrai
- Faux

Question 8

Les tests de performance doivent être réalisés le plus tôt possible dans la vie du logiciel.

- Vrai
- Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Quel indicateur qualité se rattache à la tolérance aux pannes ?

- ○ La maintenabilité
- ○ La portabilité
- ◉ La fiabilité
- ○ L'utilisabilité

Q On parle de fiabilité lorsque votre application est tolérante aux pannes. En cas de coupure d'électricité, vous pourriez avoir un onduleur qui prend le relais et continue d'alimenter la machine qui héberge votre logiciel. En cas de panne de l'interface avec la messagerie (ex : Outlook), votre logiciel ne crash pas malgré l'absence de notifications qui ne peuvent plus être envoyées.

Question 2

Votre logiciel doit migrer d'une plateforme à une autre afin de gagner en performance. Quel est le critère qualité concerné ?

- ○ La maintenabilité
- ◉ La portabilité
- ○ La capacité fonctionnelle

Q Lorsque vous pouvez déployer facilement votre logiciel sur de nouveaux environnements, on dit que votre logiciel est facilement portable. Il s'adapte facilement et possède peu de dépendances vis-à-vis du matériel et de l'environnement logiciel utilisé.

Question 3

Vous mesurez le nombre de crashs par utilisateur et par heure d'utilisation. Quel est le critère qualité concerné ?

- ◉ La fiabilité
- ○ La performance
- ○ La facilité d'exploitation

Q La quantité de crashs par heure d'utilisation est un bon indicateur, qui vous informe rapidement sur la fiabilité de votre logiciel vis-à-vis des sollicitations qui lui sont demandées par l'ensemble des utilisateurs.

Question 4

Remplir de hautes exigences pour l'ensemble des critères qualité ne peut être que bénéfique.

- ○ Vrai
- ◉ Faux

Q Il est important de choisir ses indicateurs qualité avec soin, car vous ne pourrez probablement pas tout couvrir, et le mieux est encore une fois l'ennemi du bien. Focalisez-vous sur ce qui est le plus important, en prenant en compte que des indicateurs peuvent être en concurrence les uns avec les autres (exemple : performance VS nombre de fonctionnalités).

Question 5

L'augmentation de la fiabilité peut-elle avoir un impact positif sur l'utilisabilité ?

Tous droits réservés à STUDI - Reproduction interdite

- [x] Oui
- [ ] Non

C'est un exemple de complémentarité d'indicateurs qualité. En améliorant la fiabilité de votre logiciel, vous pourriez aussi très bien améliorer son utilisabilité (facile à apprendre, comprendre, exploiter).

Exercice p. 8 Solution n°2

Question 1

Un objectif SMART est un objectif :

- [ ] Spécifique, Minutieux, Approfondi, Réaliste, Temporellement défini
- [ ] Simple, Minutieux, Automatique, Assignable, Temporellement défini
- [x] Spécifique, Mesurable, Assignable, Réaliste, Temporellement défini

Question 2

« *L'équipe doit produire 1 000 lignes de codes et 20 tests unitaires pour la prochaine release du 30 juin* ». Est-ce un objectif SMART ?

- [ ] Vrai
- [x] Faux

L'objectif n'est pas clairement assigné. Qui est responsable de cet objectif ? Pour un bon objectif, il est essentiel de désigner un pilote. Sinon, chacun va attendre de l'autre qu'il fasse le travail avant lui et il se passe que rien ne se passe. Le groupe passe tout simplement à côté de l'objectif.

Question 3

Votre chef vous demande de concevoir un logiciel fiable. Que lui répondez-vous ?

- [ ] Oui bien sûr, ce sera fait !
- [x] Fiable n'est pas un objectif SMART

Ne pensez pas que votre chef aura une meilleure estime de vous parce que vous êtes serviable et répondez toujours « *oui* ». Vous demander de concevoir un logiciel rapide, ou fiable, n'est pas un critère d'évaluation suffisant. Que cache le mot « *fiable* » ? Comment pourra-t-on mesurer l'atteinte de l'objectif ? Utilisez la méthode SMART pour définir les objectifs du projet.

Question 4

« *Le logiciel doit être utilisable par Jean-Michel d'ici vendredi prochain* ». Est-ce un objectif SMART ?

- [ ] Vrai
- [x] Faux

Cet objectif n'est ni spécifique ni mesurable. Comment peut-on savoir que le logiciel est utilisable par Jean-Michel ? Le simple fait de le lancer le rend-il utilisable ? Que se passe-t-il si Jean-Michel peut faire 90 % de son travail, mais qu'il ne peut rien terminer à cause d'une anomalie bloquante : le logiciel est-il toujours utilisable ?

Exercice p. 11 Solution n°3

Question 1

Lequel de ces tests est réalisé « *en boîte noire* » ?

- [ ] Le test unitaire
- [x] Le test de qualification

Tous droits réservés à STUDI - Reproduction interdite

- ○ Le test d'intégration
- ○ Le test d'acceptation

Le test de qualification est réalisé en boîte noire. C'est-à-dire que le test est exécuté sans que l'on connaisse le logiciel à tester, ses composants, ses interfaces, etc.

Question 2

Un test consolidé dans un script est :

- ○ Une charge
- ◉ Un investissement

Si vous lancez régulièrement les mêmes tests, c'est une charge qui pèse sur vous. Pendant que vous exécutez ces tâches répétitives, vous ne pouvez en effet pas produire. Tandis qu'en passant du temps à automatiser les tests, vous concevez un investissement qui peut vous faire gagner énormément de temps.

Question 3

La service rendu par le logiciel est vérifié par :

- ○ Des tests de robustesse
- ○ Des tests de compatibilité
- ◉ Des tests fonctionnels

Le service rendu par votre logiciel est testé au travers des tests fonctionnels. Les utilisateurs analysent le logiciel et vérifient que leurs besoins sont bien satisfaits.

Question 4

Les tests de performance sont réalisés à la fin du projet.

- ◉ Vrai
- ○ Faux

On vérifie les performances du logiciel plutôt vers la fin, après avoir apporté toutes les dernières modifications au code.

Question 5

Une mise en production à blanc permet de s'assurer de :

- ○ La robustesse du logiciel
- ◉ La compatibilité du logiciel
- ○ La bonne adéquation du logiciel avec les besoins fonctionnels

En simulant une mise en production sur un environnement identique à celui de production, vous vous assurez de la compatibilité du logiciel avec celui-ci. Vous gagnez en confiance pour le jour du déploiement réel.

Exercice p. 14 Solution n°4

Question 1

Vous souhaitez améliorer la qualité de votre logiciel. Quel est le principal pré-requis ?

- ◉ Avoir défini des indicateurs qualité au début du projet
- ○ Disposer d'utilisateurs motivés

Tous droits réservés à STUDI - Reproduction interdite

- ○ Acheter du matériel performant
- ○ Avoir un bon chef

Il faut définir ses indicateurs au début du projet, et s'efforcer de les mesurer au fil du temps à des fins de comparaison. Si vous n'avez aucune référence, vous ne pourrez pas savoir si la qualité de votre logiciel a été améliorée ou dégradée.

Question 2

Le versionning, c'est :

- ○ Le fait d'archiver en temps et en heure son code source
- ○ Afficher le numéro de version de son logiciel de manière apparente
- ◉ Adopter une convention de numérotation pour les versions de son logiciel

Le versionning vous permet de vous y retrouver facilement dans les différentes versions de votre logiciel. Il est crucial d'adopter le versionning avant de débuter le moindre test : cela vous permet de relier les anomalies et les correctifs à vos versions.

Question 3

Laquelle de ces versions est susceptible de comporter le moins d'anomalies ?

- ○ La version alpha
- ○ La version beta
- ◉ La version rc1

La version rc1 est une « *release candidate* », c'est-à-dire la version susceptible de basculer en production. C'est la version la plus aboutie de cette liste et donc celle qui logiquement devrait comporter le moins d'anomalies.

Question 4

Vous venez de déployer la version 14.1 de votre logiciel en production. On vous demande de travailler sur des évolutions importantes en vue d'une prochaine version, mais aussi sur des correctifs, car plusieurs anomalies sont identifiées. Vous livrez les correctifs. Quel numéro donnez-vous à la version ?

- ○ 15.0
- ○ 15.1
- ◉ 14.2

Vous allez itérer sur la version majeure actuelle (14) en y apportant des correctifs. Vous allez donc incrémenter son numéro de révision pour passer de 1 à 2 ; c'est-à-dire de 14.1 à 14.2.

Question 5

Vous désirez faire tester votre logiciel aussi tôt que possible. Vous réunissez pour cela une équipe d'utilisateurs motivés. Quelle est la version qui servira aux tests ?

- ○ Dev
- ◉ Alpha
- ○ Beta

La version dev n'est pas une version utilisable. Comme vous souhaitez faire tester le plus tôt possible, cela veut dire que vous utilisez la toute première version testable : c'est la version alpha.

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. Solution n°5

Vous proposez à l'équipe de faire une pause dans les spécifications techniques et de réfléchir à la direction que vous souhaitez donner à la qualité de ce logiciel.

Quels sont les facteurs qualité les plus importants dans ce projet ?

Faut-il une application toujours disponible, 24 H / 24 ? Faut-il une application robuste ? Une application facile à maintenir ? Qui va prendre en charge le maintien en condition opérationnelle une fois le développement terminé ?

Les utilisateurs sont-ils habitués à l'informatique, ou faut-il s'appliquer à mettre au point l'interface la plus simple possible ?

Faut-il apporter un soin particulier à la documentation dans le cadre d'un transfert de compétences ?

Vous proposez un brainstorming avec l'ensemble de l'équipe afin d'identifier où l'accent doit être mis en termes de qualité, et quels sont vos objectifs. Vous proposez la méthode SMART afin de construire chacun des objectifs.

Exercice p. Solution n°6

Grâce à la méthode SMART, vous avez défini des objectifs :

- Spécifique
- Mesurable
- Assignable
- Réaliste
- Temporellement défini

Vous aviez déterminé que votre logiciel doit tourner 24 H / 24, car l'usine ne s'arrête jamais et qu'il est important de pouvoir accéder à votre application à tout moment.

Dans vos objectifs, vous avez identifié entre autres, un taux de disponibilité de 98 % sur la semaine et une forte tolérance aux pannes matérielles.

Vous décidez donc de monitorer l'activité de votre logiciel. Vous enregistrez toutes les coupures de service ainsi que leurs durées.

D'après vos objectifs qualité, il ne doit pas y avoir plus de 3 H 22 de coupure de service par période de 7 jours (2 % max. d'indisponibilité par semaine). Avez-vous rempli le contrat de service ?

Que se passe-t-il si vous provoquez une coupure d'électricité ? Un onduleur prend-il le relais ? Que se passe-t-il si le serveur crash ? Y a t-un serveur miroir qui prend le relais ?

Suivez, documentez et améliorez votre logiciel jusqu'à atteindre vos objectifs !

Exercice p. 16 Solution n°7

Question 1

Des process qualité bien documentés permettent de prédire la fiabilité d'un logiciel une fois développé.

- ○ Vrai
- ◉ Faux

Q La fiabilité d'un logiciel ne peut pas être déterminée à l'avance. On peut uniquement s'efforcer d'adopter de bonnes pratiques afin de limiter le nombre d'anomalies générées.

Question 2

Parmi les phrases suivantes, laquelle est relative à la fiabilité ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Le produit répond parfaitement à la demande du client.
- ○ Le logiciel peut être maintenu facilement par les équipes techniques.
- ◉ Les utilisateurs ont confiance dans les traitements effectués par le logiciel.

Votre logiciel est réputé fiable quand les utilisateurs l'utilisent avec confiance. La perte ou l'altération de données déjà saisies sont des anomalies extrêmement dommageables pour l'image de votre logiciel et la fiabilité que perçoivent les utilisateurs.

Question 3

Une utilisation incorrecte de votre logiciel ne doit pas entraîner de dysfonctionnement. On parle ici de :

- ○ Performance
- ◉ Robustesse
- ○ Maintenabilité
- ○ Portabilité

Il est important de savoir comment votre logiciel réagit lors de circonstances imprévues, notamment lors d'une utilisation fonctionnelle différente (vous n'aviez pas prévu que les utilisateurs cliquent sur tel ou tel bouton à tel moment). Si votre logiciel se comporte normalement même après un cas d'utilisation non prévu, on dit qu'il est robuste.

Question 4

La maintenabilité, c'est :

- ○ L'aptitude du logiciel à fonctionner dans un nouvel environnement
- ◉ La mesure de l'effort nécessaire pour modifier le logiciel
- ○ La capacité du logiciel à délivrer des résultats conformes

Votre logiciel est facilement maintenable si on peut y apporter des correctifs et des évolutions de façon simple. La maintenabilité agit fortement sur les coûts d'exploitation et de maintenance.

Question 5

Une augmentation de la capacité fonctionnelle peut-elle avoir un impact négatif sur la performance de votre application ?

- ◉ Oui
- ○ Non

C'est un exemple de critères qualité concurrents. Si vous augmentez le nombre de fonctionnalités offertes par votre application (meilleure qualité fonctionnelle), vous vous exposer à potentiellement sacrifier sa performance (perte de qualité technique).

Question 6

Il vaut mieux se concentrer sur les tests unitaires au début du projet afin d'en produire le plus possible et être tranquille pour le reste du développement.

- ○ Oui, ce qui est fait n'est plus à faire !
- ◉ Non, la rédaction des tests unitaires est une tâche continue

Les tests unitaires accompagnent les logiciels pendant toute la durée de leurs développements. Ils sont vivants, et doivent s'adapter à chaque nouvelle version produite.

Tous droits réservés à STUDI - Reproduction interdite

Question 7

Le plan qualité est un document indispensable que possèdent forcément toutes les entreprises informatiques.

- ○ Vrai
- ◉ Faux

Le plan qualité est un document utile et précieux, mais qui est malheureusement peu répandu. N'hésitez pas à proposer à votre entreprise d'investir du temps afin d'en écrire un si elle n'en possède pas !

Question 8

Les tests de performance doivent être réalisés le plus tôt possible dans la vie du logiciel.

- ○ Vrai
- ◉ Faux

Les tests de performance sont réalisés à la fin du projet, lorsque toutes les fonctionnalités sont implémentées, que le périmètre est verrouillé. L'optimisation arrive en dernier. Sinon, vous passerez votre temps à optimiser du code voué à disparaître ou à être modifié dans des versions futures.

Tous droits réservés à STUDI - Reproduction interdite